FOODRESCUE / FRONTEND LOGIC

Logic checklist, explained

Seven frontend ideas, shown in the app and the source code behind each one.

Live site on this device: http://127.0.0.1:5173/

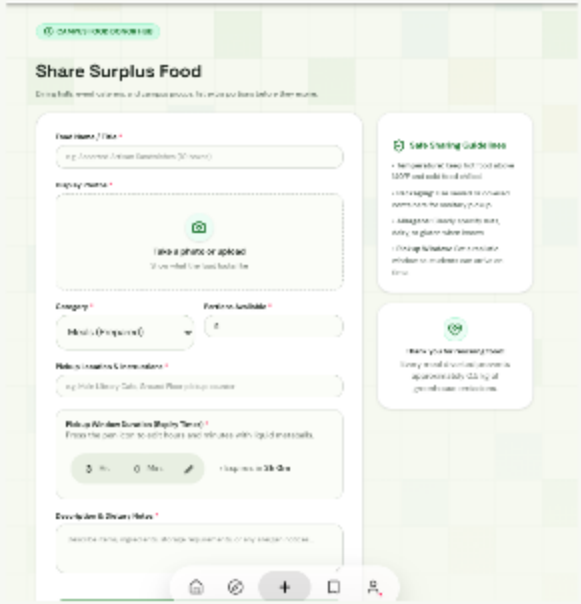
Built-in Student demo. Sample listings only. No posts or claims were submitted while capturing.

CHECKLIST ITEM 01 OF 07

Functions

A function groups the steps for one task.

IN THE APP



Screen: /app/post

SOURCE CODE

apps/frontend/src/modules/post/PostPage.jsx — lines 81–91

```
81        const totalHours = (validation.values.duration.hours || 0) +
    (validation.values.duration.minutes || 0) / 60;
82        const result = await createFoodPost({
83          foodName: validation.values.foodName,
84          description: validation.values.description,
85          category: validation.values.category,
86          quantity: validation.values.quantity,
87          pickupAddress: validation.values.pickupAddress,
88          photoUrl: photos[0],
89          expiresHours: totalHours > 0 ? totalHours : 3
90        });
91
```

Source: PostPage.jsx, lines 81 to 91

PRESENTER CUE

Here, handleSubmit validates the draft, calculates the expiry, and calls createFoodPost.

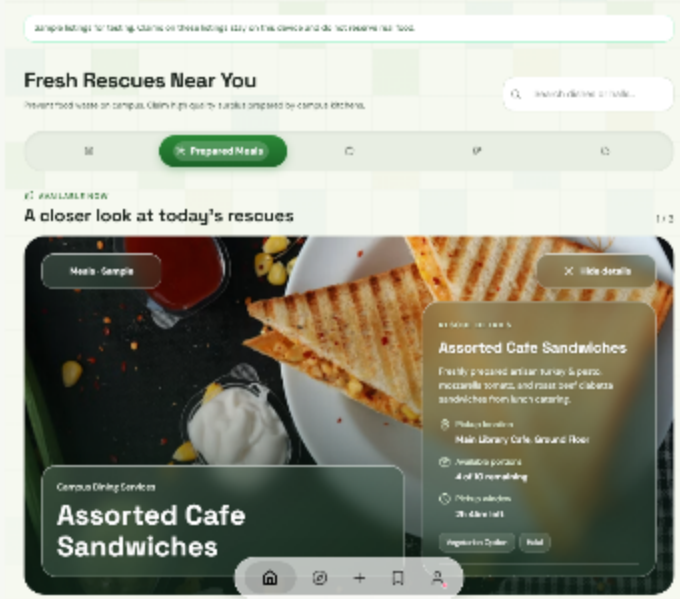
Built-in demo capture. No post or claim was submitted while preparing this deck.

CHECKLIST ITEM 02 OF 07

Control structures

Control structures decide which items continue to the screen.

IN THE APP



Screen: /app/home-feed

SOURCE CODE

apps/frontend/src/modules/feed/HomeFeed.jsx — lines 67–74

```
67   const filtered = posts.filter((item) => {
68     const matchCat = category === 'All' || item.category?.toLowerCase() ===
   category.toLowerCase();
69     const matchSearch = !searchQuery ||
70       item.food_name?.toLowerCase().includes(searchQuery.toLowerCase()) ||
71       item.pickup_address?.toLowerCase().includes(searchQuery.toLowerCase());
72     return matchCat && matchSearch;
73   });
74
```

Source: HomeFeed.jsx, lines 67 to 74

PRESENTER CUE

A post stays visible only when it passes both the category test and the search test.

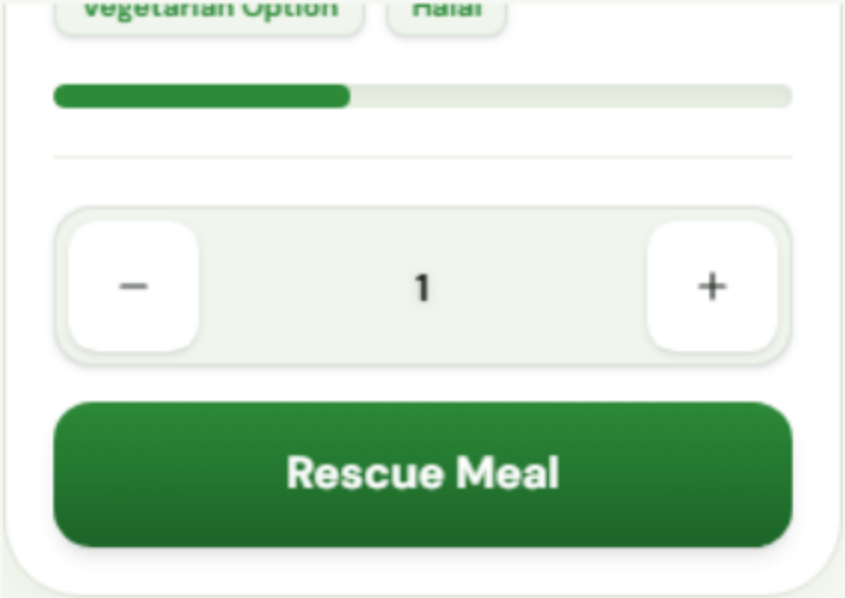
Built-in demo capture. No post or claim was submitted while preparing this deck.

CHECKLIST ITEM 03 OF 07

Event handlers

An event handler connects a user action to program logic.

IN THE APP



Screen: /app/home-feed

SOURCE CODE

apps/frontend/src/components/food/FoodCard.jsx — lines 76–87

```
76   async function handleClaim(e) {
77     e.stopPropagation();
78     const validationError = validateClaimQuantity(post, portion);
79     if (validationError) {
80       setClaimError(validationError);
81       return;
82     }
83
84     setIsClaiming(true);
85     setClaimError('');
86     try {
87       const claim = await claimFoodPost(post, portion);
```

Source: FoodCard.jsx, lines 76 to 87

PRESENTER CUE

The claim control calls handleClaim, which checks the quantity before starting the claim request.

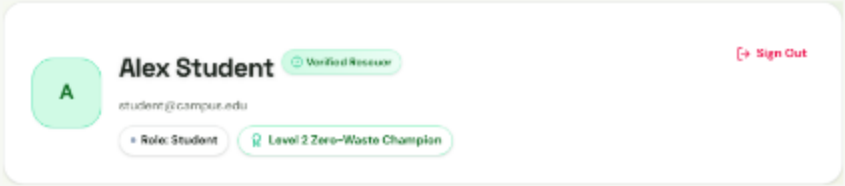
Built-in demo capture. No post or claim was submitted while preparing this deck.

CHECKLIST ITEM 04 OF 07

Data passed between pages

Props let a parent component share data with a child page.

IN THE APP



Screen: /app/profile

SOURCE CODE

apps/frontend/src/App.jsx — lines 79–87

```
79            <Route exact path="/app/home-feed">
80              <HomeFeed userProfile={userProfile} />
81            </Route>
82            <Route exact path="/app/map">
83              <MapPage userProfile={userProfile} />
84            </Route>
85            <Route exact path="/app/post">
86              <PostPage userProfile={userProfile} />
87            </Route>
```

Source: App.jsx, lines 79 to 87

PRESENTER CUE

App passes userProfile into the screens, and the profile card reads its name, email, and role.

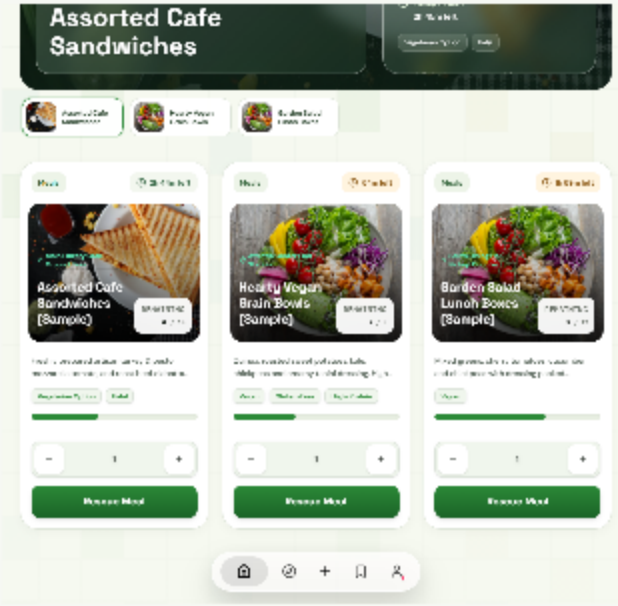
Built-in demo capture. No post or claim was submitted while preparing this deck.

CHECKLIST ITEM 05 OF 07

Lists displayed

A list turns an array of posts into repeated interface elements.

IN THE APP



Screen: /app/home-feed

SOURCE CODE

apps/frontend/src/modules/feed/HomeFeed.jsx — lines 209–217

```
209            {filtered.map((post) => (
210              <FoodCard
211                key={post.id}
212                post={post}
213                previewOpen={previewPostId === post.id}
214                onPreviewClose={() => setPreviewPostId(null)}
215                onClaimSuccess={handleClaimSuccess}
216              />
217            ))}
```

Source: HomeFeed.jsx, lines 209 to 217

PRESENTER CUE

map renders one FoodCard per filtered post, while the post ID helps React track each item.

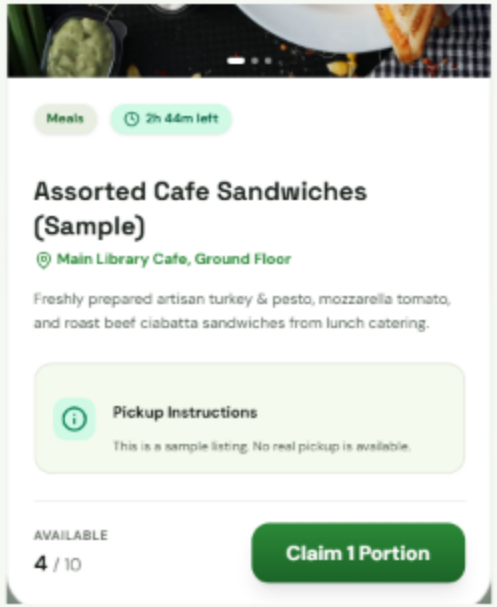
Built-in demo capture. No post or claim was submitted while preparing this deck.

CHECKLIST ITEM 06 OF 07

Reusable services

A service separates data access from the screen that displays the result.

IN THE APP



Screen: /app/home-feed

SOURCE CODE

apps/frontend/src/services/food.js — lines 22–30

```
22 export async function getFoodPosts() {
23   if (isSupabaseConfigured()) {
24     const { data, error } = await supabase
25       .from('food_posts')
26       .select('*, donor:donor_id(full_name, phone)')
27       .eq('status', 'active')
28       .order('created_at', { ascending: false });
29     if (error) throw error;
30     if (!Array.isArray(data)) throw new Error('bad response');
```

Source: services/food.js, lines 22 to 30

PRESENTER CUE

HomeFeed calls getFoodPosts; the service handles retrieving active listings and the demo fallback.

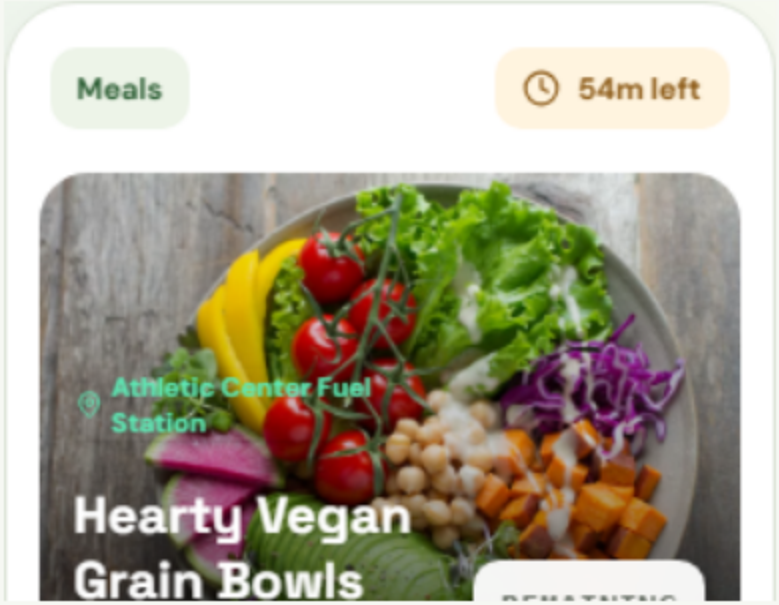
Built-in demo capture. No post or claim was submitted while preparing this deck.

CHECKLIST ITEM 07 OF 07

Clear names and comments

Descriptive names and comments explain what a calculation is doing.

IN THE APP



Screen: /app/home-feed

SOURCE CODE

apps/frontend/src/components/food/FoodCard.jsx — lines 66–74

```
66   // Time remaining calculation
67   const expiresAt = new Date(post.expires_at).getTime();
68   const now = Date.now();
69   const diffMinutes = Math.max(0, Math.floor((expiresAt - now) / (60 * 1000)));
70   const hoursLeft = Math.floor(diffMinutes / 60);
71   const minsLeft = diffMinutes % 60;
72
73   const isUrgent = diffMinutes < 45;
74   const isModerate = diffMinutes < 120;
```

Source: FoodCard.jsx, lines 66 to 74

PRESENTER CUE

expiresAt, diffMinutes, hoursLeft, and minsLeft make the countdown logic easier to follow.